

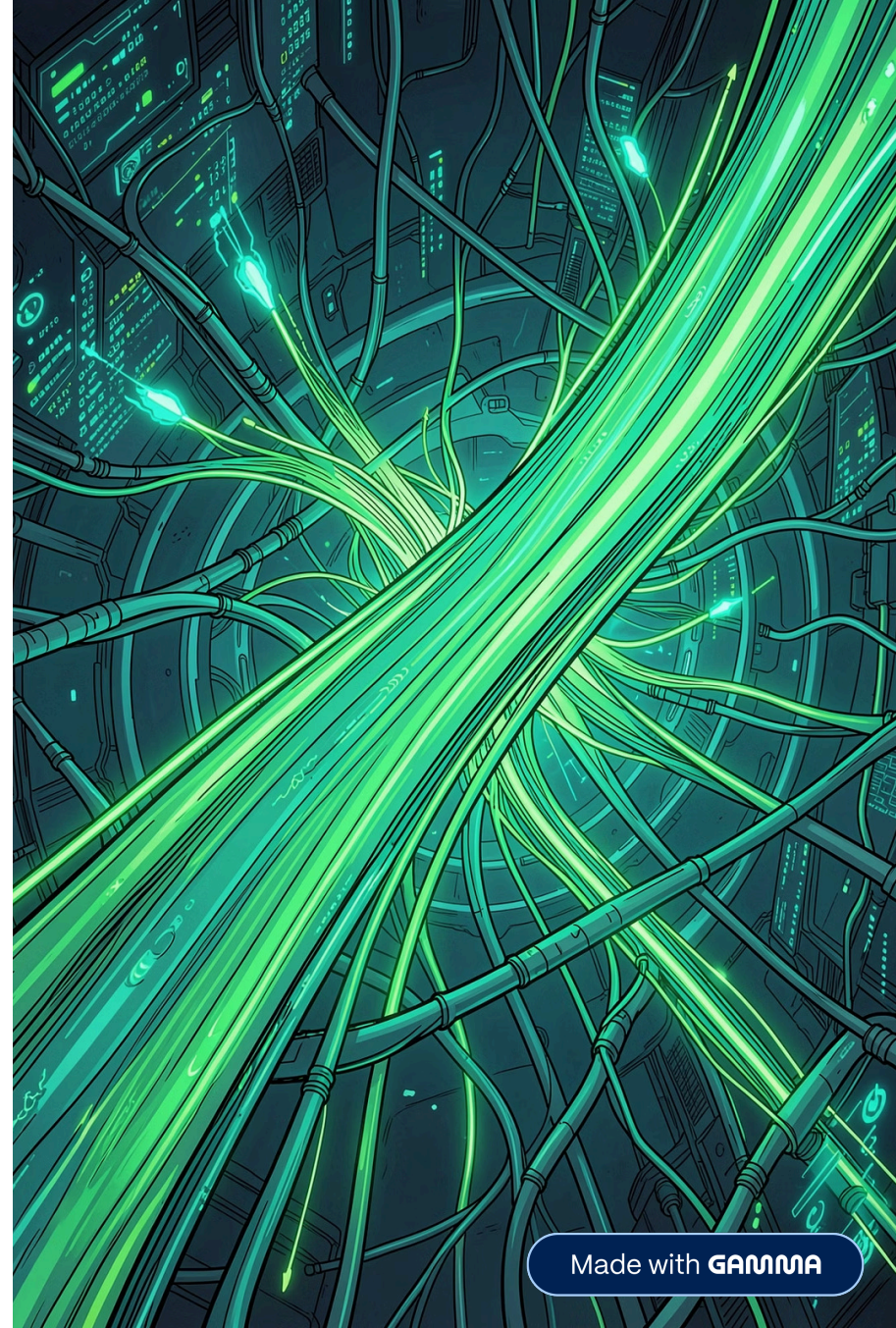
Dalla Teoria al Codice

Un Sistema

Predittivo Completo

Un progetto Python che combina machine learning, Flask e Docker per trasformare dati grezzi in previsioni affidabili.

A cura di Combi Federico, Brivio Emanuel





La Consegna: Estrarre Valore dai Dati

Il Dataset

Il celebre dataset del Titanic da Kaggle: un benchmark classico per la classificazione binaria con variabili demografiche e strutturali.

L'Obiettivo

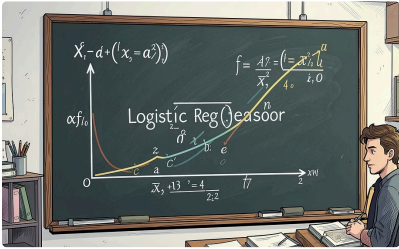
Trasformare dati grezzi in previsioni accurate sulla sopravvivenza, bilanciando performance e leggibilità del modello.

La Variabile Target

Stimare se il passeggero riesce a sopravvivere o meno

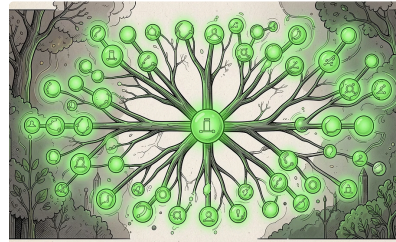
Modelli di machine learning

Tre Approcci a Confronto



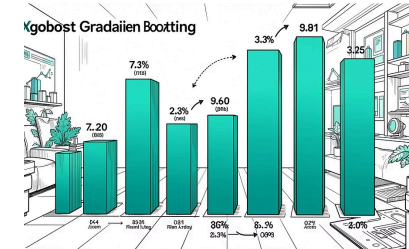
Regressione Logistica

La base statistica per la classificazione binaria. Interpretabile, veloce e ideale come punto di partenza.



Random Forest

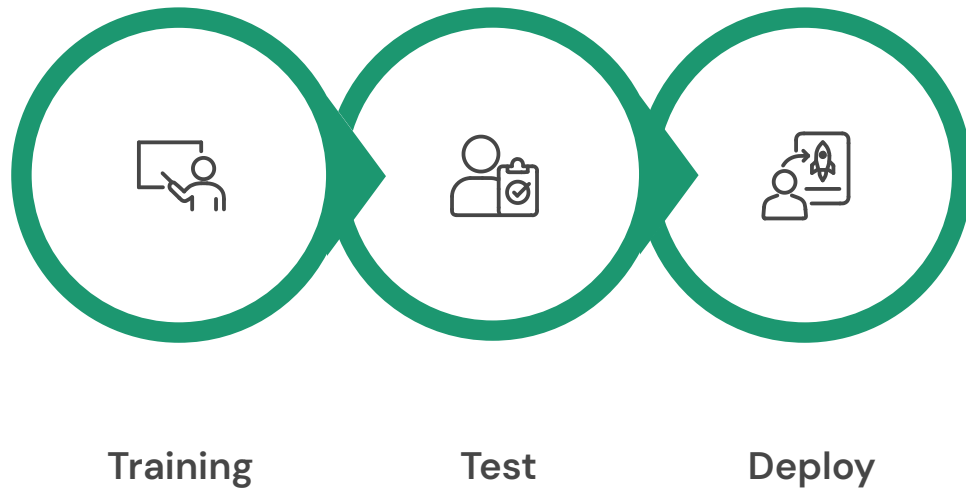
Collezione di alberi decisionali che combina più predizioni per ridurre overfitting e aumentare robustezza.



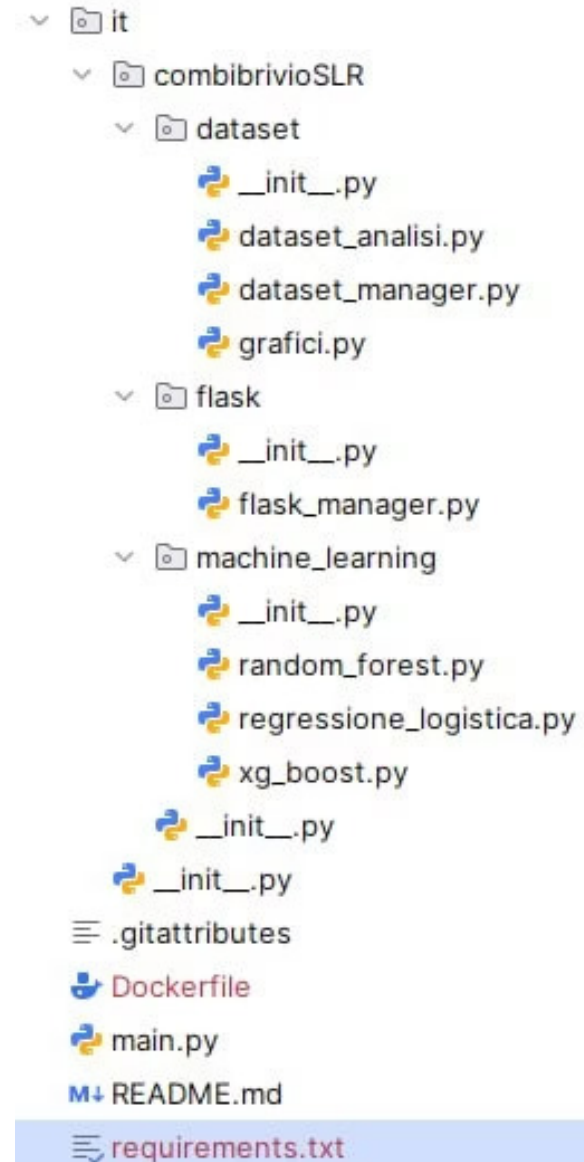
XGBoost

Gradient boosting ottimizzato: velocità e precisione superiori grazie alla regolarizzazione e al parallelismo.

Architettura del Progetto



Il progetto è realizzato in Python, Pandas gestisce la manipolazione dei dati e Scikit-learn fornisce gli strumenti di modellazione e valutazione.



Analisi Esplorativa dei Dati (EDA)

Un'Immersione Profonda nel Dataset del Titanic



Struttura del Dataset

Analisi iniziale delle dimensioni e dei tipi di dato. Identificate 891 righe e 12 colonne, tra numeriche e categoriche.



Gestione Valori Mancanti

Affrontati i valori mancanti: 'Age' (imputazione con la mediana), 'Fare' (imputazione con la media), 'Embarked' (imputazione con la moda), 'Cabin' (droppata visto che 687/891).



Dropping colonne irrilevanti

Eliminate le colonne 'Name', 'Ticket', 'PassengerId' visto che non influiscono sulla predizione. Questi valori rappresentano il Nome, N_biglietto, ID_passeggero

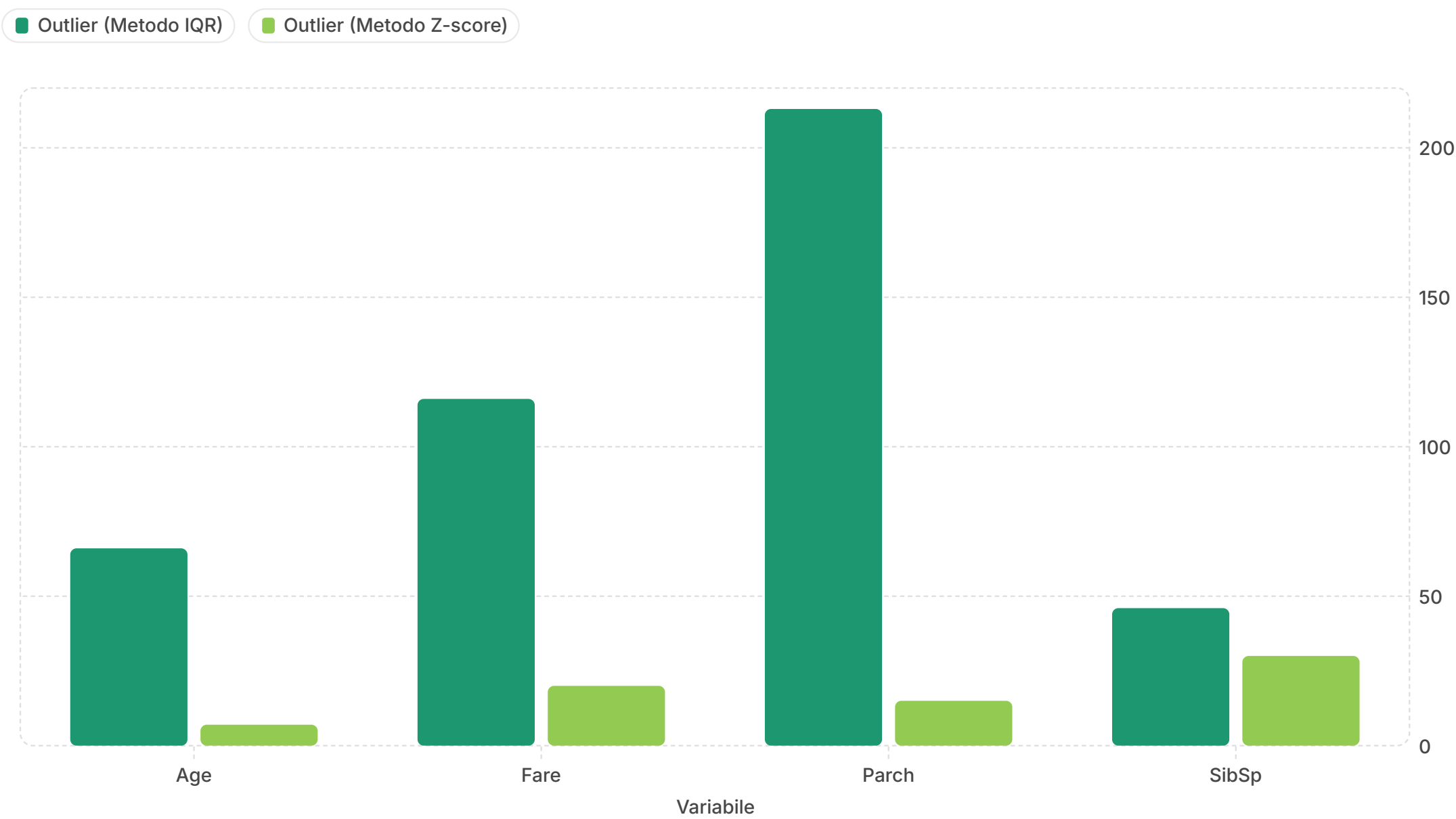


Encoding

Encoding di 'Sex' in modo da avere male = 1 e female = 0.
Encoding di 'Embarked' separandolo nelle 3 colonne possibili

Analisi degli Outlier

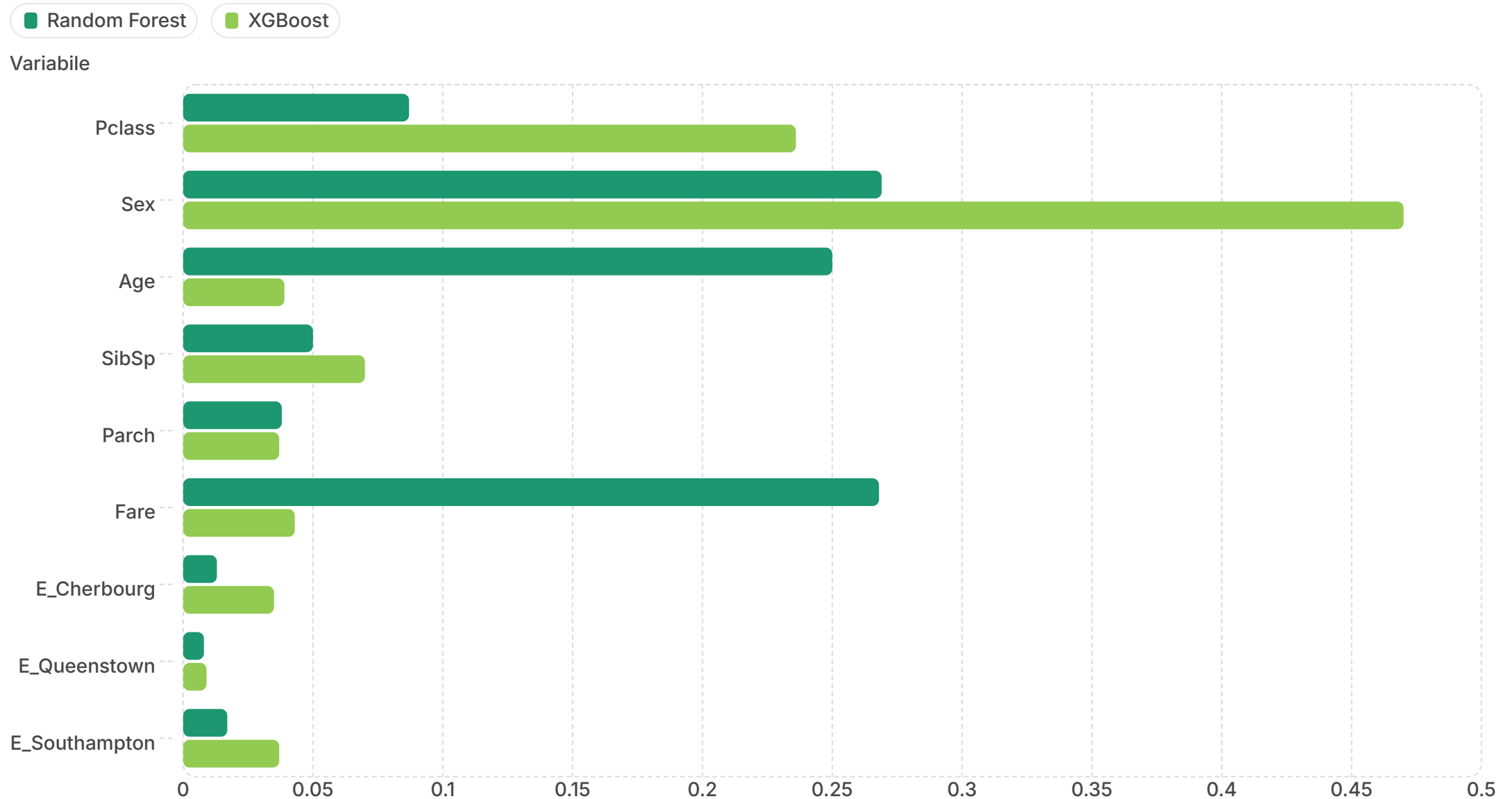
Per garantire la robustezza del modello, abbiamo analizzato e quantificato la presenza di outlier nelle variabili numeriche utilizzando due metodi principali: l'Interquartile Range (IQR) e lo Z-score. Il grafico seguente riassume il numero di outlier identificati per ciascuna variabile:



Come si evince, il metodo IQR tende a identificare un numero maggiore di outlier rispetto allo Z-score, specialmente per variabili come 'Parch' e 'Fare', evidenziando la loro distribuzione asimmetrica. Le variabili categoriche ('Pclass', 'Sex', 'Survived') non presentano outlier in entrambi i contesti.

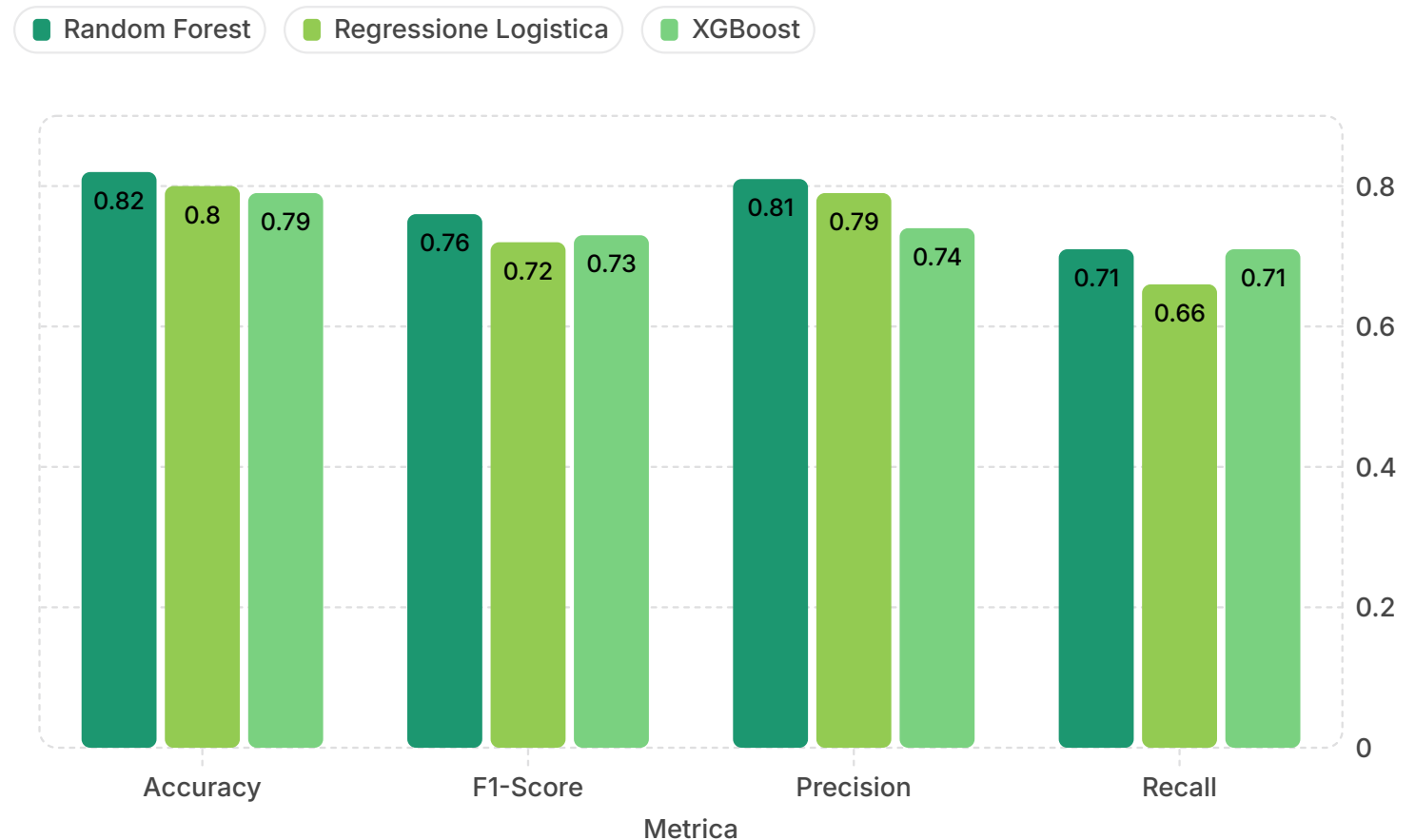
Importanza delle variabili

Questo grafico mostra come ciascun modello assegna peso alle diverse feature nelle proprie previsioni, evidenziando differenze tra Random Forest, Regressione Logistica e XGBoost.



Analisi Comparativa dei Modelli

Ogni modello è stato valutato su quattro metriche chiave per identificare il più performante e comprendere i trade-off tra complessità e velocità.



Accuracy

indica quante predizioni sono corrette in totale in %

Precisione

quanti positivi predetti sono effettivamente positivi

F1-Score

quanto il modello è bilanciato tra "non sbagliare positivi" e "trovarli tutti"

Interpretazione delle valutazioni dei modelli

Le prestazioni dei modelli sono molto vicine tra loro e suggeriscono un problema sostanzialmente **quasi lineare**, con poche variabili davvero decisive.

Prestazioni dei Modelli

- Random Forest: **82.68%** (*best model*)
- Regressione Logistica: **80.45%** (*very close*)
- XGBoost: **79.33%** (*slightly lower*)
- Le differenze sono contenute.

Feature più importanti

- **Sex** è la variabile dominante in tutti i modelli.
- Random Forest distribuisce l'importanza tra **Sex**, **Fare** e **Age**.
- XGBoost si concentra soprattutto su **Sex** e **Pclass**.

Interpretabilità della Logistica

- **Male** → forte riduzione della probabilità di sopravvivenza.
- **Pclass** è molto rilevante.
- **Age** ha effetto negativo: più età → meno sopravvivenza.
- **Fare** è positivo ma debole.

Altre feature

- Multicollinearità tra **Fare** e **Pclass**.
- **Embarked** ha un impatto trascurabile.

Flask e Docker: Dal Modello all'Applicazione

Flask – L'Interfaccia Web

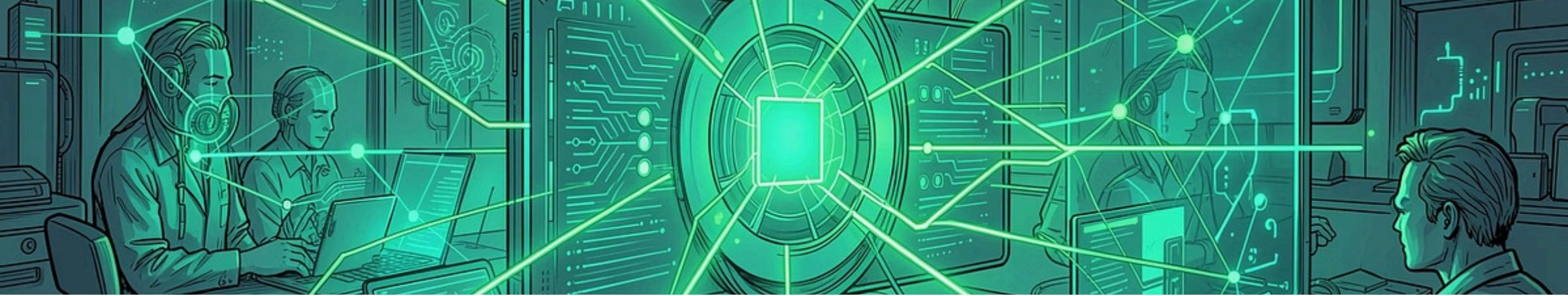
Flask è il framework Python che crea l'interfaccia web per interagire con il modello.

- Attraverso questa interfaccia, gli utenti possono inserire i dati dei passeggeri, ottenere le previsioni generate dai diversi modelli e consultare il dataset per esplorarlo.

Docker – La Containerizzazione

Docker incapsula l'intera applicazione, modello e Flask, in un container.

- L'applicazione funziona identicamente su qualsiasi dispositivo o server.
- Deployment semplificato e scalabilità garantita.



Sviluppi futuri

Cross-validation

Applicare la cross-validation per ottenere una valutazione più robusta dei modelli, dato il numero limitato di osservazioni.

Tuning degli iperparametri

Effettuare il tuning per migliorare l'accuratezza dei diversi modelli.

Ulteriori modelli

Integrare nuovi modelli di machine learning per ampliare il confronto e le possibilità di previsione.

Deployment online

Rendere il container accessibile online, non solo in rete locale (LAN), per consentire un utilizzo da remoto.